ACM International Collegiate Programming Contest 1999-2000

Sponsored by IBM

Southwestern European Regional Contest

Supported by Telefónica I+D and Divisa Informática S.A.

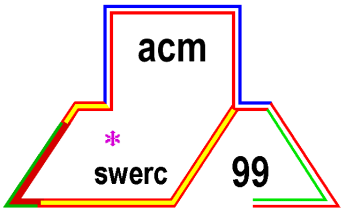


University of Valladolid, Spain

November 21th, 1999

Problem Set

This problem set should contain eight (8) problems on fifteen (15) numbered pages.
Please inform a runner immediately if something is missing from your problem set.



Problem A
Date Bugs

There are rumors that, there are a lot of computers having a problem with the year 2000. As they use only two digits to represent the year, the date will suddenly turn from 1999 to 1900. In fact, there are also many other, similar problems. On some systems, a 32-bit integer is used to store the number of seconds that have elapsed since a certain fixed date. In this way, when 2^{32} seconds (about 136 Years) have elapsed, the date will jump back to whatever the fixed date is.

Now, what can you do about all that mess? Imagine you have two computers C_1 and C_2 with two different bugs: One with the ordinary Y2K-Bug (i. e. switching to 1900 instead of 2000) and one switching to 1904 instead of 2040. Imagine that the C_1 displays the year 1941 and C_2 the year 2005. Then you know the following (assuming that there are no other bugs): The real year can't be 1941, since, then, both computers would show the (same) right date. If the year would be 2005, the C_1 would show 1905, so this is impossible, too. Looking only at C_1, we know that the real year is one of the following: 1941, 2041, 2141, etc. We now can calculate what C_2 would display in these years: 1941, 1905, 2005, etc. So in fact, it is possible that the actual year is 2141.

To calculate all this manually is a lot of work. (And you don't really want to do it each time you forgot the actual year.) So your task is to write a program which does the calculation for you: find the first possible real year after 1937, knowing what some other computers claim and knowing their bugs.[1]

Input

The input file contains several test cases, in which the actual year has to be calculated. The description of each case starts with a line containing an integer n ($1 \leq n \leq 20$), the number of computers. Then, there is one line containing three integers y, a, b for each computer ($0 \leq b \leq y < a < 10000, b \leq 1937$). y is the year the computer displays, a is the year in which the bug happens (i. e. the first year which can't be displayed by this computer) and b is the year that the computer displays instead of a.

The input is terminated by a line containing a zero.

Output

For each test case, output the line "`Case #`k`:`", where k is the number of the situation. Then, output the line "`The actual year is` z`.`", where z is the smallest possible year (satisfying all computers and being greater or equal to 1937). If there is no such year less than 10000, output "`Unkown bugs detected.`".

Output a blank line after each case.

[1] 1937 is the year in which the Z1 computer was build by K. Zuse. Did the Z1 already have the Y2K bug? We don't know.

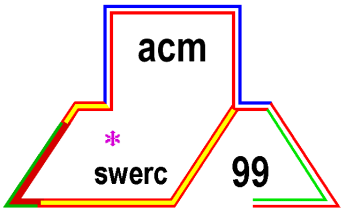
Sample Input

```
1
1999 2000 1900
2
132 200 100
135 200 100
0
```

Sample Output

```
Case #1:
The actual year is 1999.

Case #2:
Unknown bugs detected.
```



Problem B

The Archeologists' Dilemma

An archeologist seeking proof of the presence of extraterrestrials in the Earth's past, stumbles upon a partially destroyed wall containing strange chains of numbers. The left-hand part of these lines of digits is always intact, but unfortunately the right-hand one is often lost by erosion of the stone. However, she notices that all the numbers with all its digits intact are powers of 2, so that the hypothesis that all of them are powers of 2 is obvious. To reinforce her belief, she selects a list of numbers on which it is apparent that the number of legible digits is strictly smaller than the number of lost ones, and asks you to find the smallest power of 2 (if any) whose first digits coincide with those of the list.

Thus you must write a program such that given an integer, it determines (if it exists) the smallest exponent E such that the first digits of 2^E coincide with the integer (remember that more than half of the digits are missing).

Input

It is a set of lines with a positive integer N not bigger than 2147483648 in each of them.

Output

For every one of these integers a line containing the smallest positive integer E such that the first digits of 2^E are precisely the digits of N, or, if there is no one, the sentence "`no power of 2`".

Sample Input

```
1
2
10
```

Sample Output

```
7
8
20
```

Problem C
The Vindictive Coach

The coach of a football team, after suffering for years the adverse comments of the media about his tactics, decides to take his revenge by presenting his players in a line-up in such a way that the TV cameras would be compelled to zigzag in a ridiculous bobbing motion, by alternating taller and shorter players. However, the team captain objects that he must be the first of the line by protocolary reasons,and that he wants to be seen in the best possible light: that is, he should not have a taller colleague nest to him unless there is no alternative (everyone else is taller than him). Even in this case, the height difference should be as small as possible, while maintaining the zigzag arrangement of the line.

With this condition the coach addresses an expert in computation (i.e. you) to help him find the number of different alignments he may make, knowing that all players have a different height. They are always numbered by stature starting by 1 as the shortest one. Of course the number of players may be arbitrary, provided it does not exceed 22.

Input

It is a set of lines, each of which contains two positive integers N and m separated by a blank space. $N(\leq 22)$ represents the number of players in the line-up and m the captain's number, who as told is always the first of the line.

Output

For every line of the input a line with positive integer indicating the number of possible alignments under the above conditions.

Sample Input

```
3 1
3 3
4 1
```

Sample Output

```
1
1
1
```

Problem D

Triple Ties: The Organizer's Nightmare

N participants compete in a chess tournament, every player competing against each other one single time. The results are registered in a square matrix in the following way:

- If player i beats player j, the digit 1 must be written at the position (i, j), where i denotes the row and j de column, and the digit 0 at the place (j, i).
- If the game ends in a draw, both numbers are zero.
- The digits at the main diagonal are also zero.

To establish the final clasification the organizer must face the problem of possible ties. The general criterion is that ties should be solved by counting the points obtained in the matches between the tied players, but sometimes this does not solve the problem. This happens for two players if they reached a draw in their respective match; for three tied players, there exist two different undecidable situations:

- If the three matches were draws.
- If there is a cyclic chain of victories, such as 1 beats 2, 2 beats 3, 3 beats 1.

Since the organizer wishes to establish different classifications according to age groups, nationality, etc. of the participants, it has been decided to elaborate a table containing all the triple ties. If all three matches end in draws, the three numbers must be annotated in increasing order, and if there is a cycle of victories, in the following order: the first in the line beats the second, and the second beats the third one. Among the three possibilities of expressing this fact, the line must be written in increasing or decreasing order. These three-digit chains must themselves be listed in lexicographical order.

Input

Unlimited number of tests. Each test consists in a line with a positive integer N representing the number of players and N lines containing the results matrix of the tournament with consecutive digits separated by a blank space ($3 \leq N \leq 100$).

Output

For every input a line containing the total amount M of undecidable triple ties; and, if $M > 0$, M lines, each of them with the three numbers of the involved players separated by a blank space and satisfying the order requirements, both within the line and in the set of lines.

Sample Input

```
3
0 0 1
1 0 0
0 1 0
3
0 1 0
0 0 0
0 0 0
3
0 0 0
0 0 0
0 0 0
```

Sample Output

```
1
3 2 1
0
1
1 2 3
```

Problem E
Colour Hash

This puzzle consists of two wheels. Both wheels can rotate both clock and counter-clockwise. They contain 21 coloured pieces, 10 of which are rounded triangles and 11 of which are separators. Figure 1 shows the final position of each one of the pieces. Note that to perform a one step rotation you must turn the wheel until you have advanced a triangle and a separator.

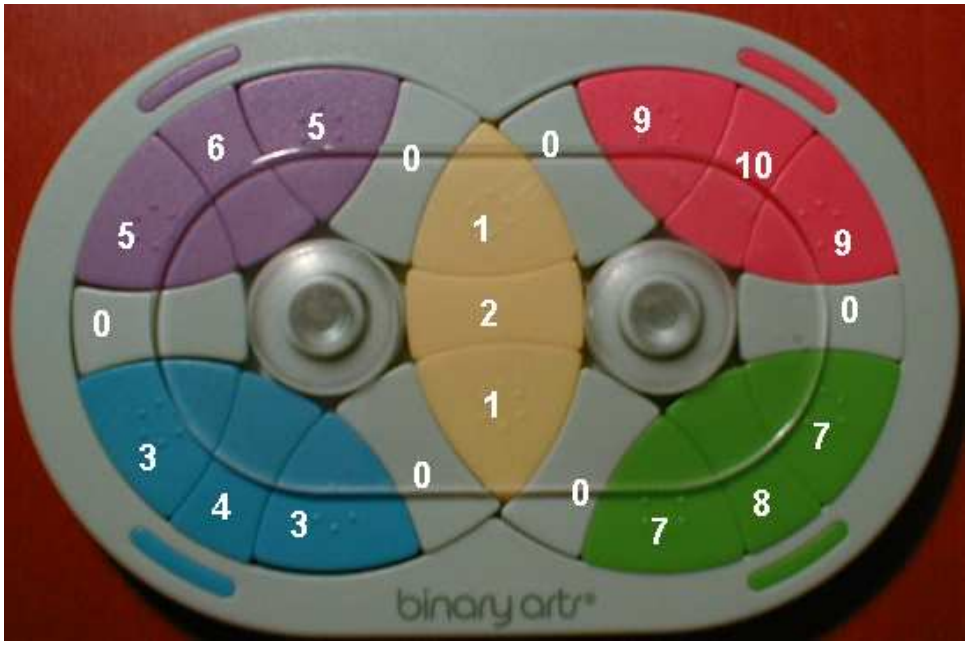


Figure 1. Final puzzle configuration

Your job is to write a program that reads the puzzle configuration and prints the minimum sequence of movements required to reach the final position. We will use the following integer values to encode each type of piece:

0	grey separator
1	yellow triangle
2	yellow separator
3	cyan triangle
4	cyan separator
5	violet triangle
6	violet separator
7	green triangle
8	green separator
9	red triangle
10	red separator

A puzzle configuration will be described using 24 integers, the first 12 to describe the left wheel configuration and the last 12 for the right wheel. The first integer represents the bottom right separator of the left wheel and the next eleven integers describe the left wheel clockwise. The thirteenth integer represents the bottom left separator of right wheel and the next eleven integers describe the right wheel counter-clockwise.

The final position is therefore encoded like:

```
0 3 4 3 0 5 6 5 0 1 2 1 0 7 8 7 0 9 10 9 0 1 2 1
```

If for instance we rotate the left wheel clockwise one position from the final configuration (as shown in Figure 2) the puzzle configuration would be encoded like:

```
2 1 0 3 4 3 0 5 6 5 0 1 0 7 8 7 0 9 10 9 0 5 0 1
```



Figure 2. The puzzle after rotating the left wheel on step clockwise from the final configuration.

Input

Input for your program consists of several puzzles. The first line of the input will contain an integer n specifying the number of puzzles. There will then be n lines each containing 24 integers separated with one white space, describing the initial puzzle configuration as explained above.

Output

For each configuration your program should output one line with just one number representing the solution. Each movement is encoded using one digit from 1 to 4 in the following way:

1 Left Wheel Clockwise rotation
2 Right Wheel Clockwise rotation
3 Left Wheel Counter-Clockwise rotation
4 Right Wheel Counter-Clockwise rotation

No space should be printed between each digit. Since multiple solutions could be found, you should print the solution that is encoded as the smallest number. The solution will never require more than 16 movements.

If no solution is found you should print "`NO SOLUTION WAS FOUND IN 16 STEPS`". If you are given the final position you should print "`PUZZLE ALREADY SOLVED`".

Sample Input

```
3
0 3 4 3 0 5 6 5 0 1 2 1 0 7 8 7 0 9 10 9 0 1 2 1
0 3 4 5 0 3 6 5 0 1 2 1 0 7 8 7 0 9 10 9 0 1 2 1
0 9 4 3 0 5 6 5 0 1 2 1 0 7 8 7 0 9 10 3 0 1 2 1
```

Sample Output

```
PUZZLE ALREADY SOLVED
1434332334332323
NO SOLUTION WAS FOUND IN 16 STEPS
```

Problem F
Maze

By filling a rectangle with slashes (/) and backslashes (\), you can generate nice little mazes. Here is an example:



As you can see, there are no branchings, so the whole maze only consists of cyclic paths and of paths entering somewhere and leaving somewhere else. We are only interested in the cycles. In our example, there are two of them.

Your task is to write a program that counts the cycles and finds the length of the longest one. The length is defined as the number of small squares the cycle consists of (the ones bordered by gray lines in the picture). In this example, the long cycle has length 16 and the short one length 4.

Input

The input consists of several test cases. Each case begins with one line containing the width and the height of the maze, followed by the maze itself. After the last case, there is a line containing "`0 0`".

Output

For each maze, first output the line "`Maze #`n`:`", where n is the number of the maze. Then, output the line "k `Cycles; the longest has length` l`.`", where k is the number of cycles and l the length of the longest one. If the maze doesn't contain any cycles, output the line "`There are no cycles.`".

Output a blank line after each test case.

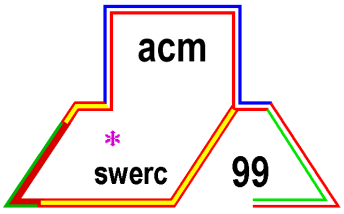
Sample Input

```
2 5
\\
\\
/\
\/
/\
4 4
/\/\
\\//
//\\
\/\/
0 0
```

Sample Output

```
Maze #1:
1 Cycles; the longest has length 4.

Maze #2:
1 Cycles; the longest has length 20.
```



Problem G
LC-Display

A friend of you has just bought a new computer. Until now, the most powerful computer he had ever used was a pocket calculator. Now, looking at his new computer, he is a bit disappointed, because he liked the LC-display of his calculator so much. So you decide to write a program that displays numbers in an LC-display-like style.

Input

The input file contains several lines, one for each number to display. Each line consists of two integers s, n ($1 \leq s \leq 10, 0 \leq n \leq 99\,999\,999$), where n is the number to be displayed and s is the size in which it shall be displayed.

The input file will be terminated by a line containing two zeros.

Output

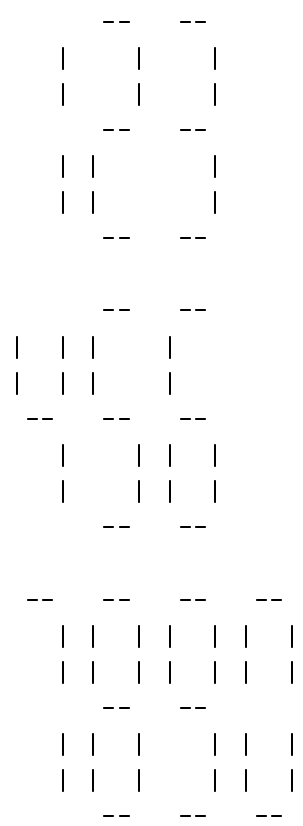
Output the numbers given in the input file in an LC-display-style using s "-" signs for the horizontal segments and s "|" signs for the vertical ones. Each digit occupies exactly $s + 2$ columns and $2s + 3$ rows. (Be sure to fill all the white space occupied by the digits with blanks, also for the last digit.) There shall be exactly one column of blanks between two digits. Output one blank line after each number.

You will find a sample of each digit in the sample input.

Sample Input

```
2 123
2 456
2 7890
0 0
```

Sample Output



Problem H
Robbery

Inspector Robstop is angry. Again, a bank has been robbed and the robber has not been caught – and this happened already the third time this year. Although he did everything in his power to catch the robber: as fast as possible, all roads leading out of the city have been blocked, so the robber could not escape. Then, the inspector asked all the people in the city to watch out for the robber, but the only messages he got were "We don't see him."

This time, it is enough! Inspector Robstop decides to analyse how the robber could escape. To do that, he asks you to write a program which takes all the information about the robber the inspector could get in order to find out where the robber has been at which time.

By chance, the city in which the bank was robbed is rectangular. The city is locked for a certain time t, and during this time, several observations "The robber isn't in the rectangle R_i at time t_i" are reported. Assuming that the robber can move at most one unit per time step (whatever the units are), your program shall try to find the exact position of the robber in each time step.

Input

The input file contains the description of several robberies. The first line of each description consists of three numbers W, H, t ($1 \leq W, H, t \leq 100$) where W is the width, H the height and t the time during which the city is locked.

The next line consist of one integer n ($0 \leq n \leq 1000$), the number of messages the inspector got. The next n lines (one for each of the messages) consist of five integers t_i, L_i, T_i, R_i, B_i each. The integer t_i is the time at which the observation has been made ($1 \leq t_i \leq t$), and L_i, T_i, R_i, B_i are the left, top, right and bottom respectively of the (rectangular) area which has been observed. ($1 \leq L_i \leq R_i \leq W, 1 \leq T_i \leq B_i \leq H$; the point $(1, 1)$ is the upper left corner of the city.) The message always tells that the robber has not been in the given rectangle at time t_i.

Output

For each robbery, first output the line "`Robbery #`k`:`", where k is the number of the robbery. Then, there are three possibilities:

If it is impossible that the robber is still in the city considering the messages, output the line "`The robber has escaped.`"

Output one line of the form "`Time step` τ`: The robber has been at` x`,`y`.`" for each time step, in which the exact location can be deduced. (x and y are the column resp. row of the robber in time step τ.) Output these lines ordered by time.

If nothing can be deduced, output the line "`Nothing known.`" and hope that the inspector won't get yet more angry.

Output a blank line after each processed case.

Sample Input

```
5 1 6
2
1  1 1  4 1
5  2 1  5 1
2 2 5
0
```

Sample Output

```
Robbery #1
Time step 1: The robber has been at 5,1.
Time step 2: The robber has been at 4,1.
Time step 3: The robber has been at 3,1.
Time step 4: The robber has been at 2,1.
Time step 5: The robber has been at 1,1.

Robbery #2
Nothing known.
```